

# Fronthaul Load Management in 5G C-RAN / O-RAN

Matrix decomposition to compress the data, the cost of matrix inversion, and sharing one link between radio units

B.Tech. Project — mid-term evaluation

September 2026

*All results are Python simulations on the 3GPP TDL-A channel model (no hardware, no measured traffic)*

Tags on every slide: [repo] was already in the repository [new] built and checked in this iteration [fix] corrected [prelim] early result [future] planned

# Why the fronthaul link is the bottleneck

*In the O-RAN 7.2x split the radio unit sends one complex number per antenna, per subcarrier, per OFDM symbol*

## How much data one radio unit (RU) produces

bits per symbol =  $2 \times M$  antennas  $\times$   $N$  subcarriers  $\times$   $b$  bits

$M = 64$ ,  $N = 1200$  (15 kHz spacing, 18 MHz),  $b = 16$  bit

→ **2.46 Mbit per symbol, 14 symbols per ms → 34.4 Gbit/s per RU**

→ **8 RUs on one link: 275 Gbit/s**

## Three ways to reduce the load

Compress the  $64 \times 1200$  matrix (fewer bits, or send only its important part)

Send fewer spatial streams than antennas, or move processing into the RU (functional split)

Share the link: give each RU a share so the total fits

## What 'load management' means here

For every OFDM symbol and every RU, pick a compression setting (and stream count) so that

**total rate of all RUs  $\leq$  link capacity**

and the reconstructed matrices are as accurate as possible.

## Not modelled (so not claimed):

MAC scheduling, retransmissions, latency, real hardware timing, measured traffic

**The problem: several RUs, one link, a fixed number of bits per symbol — decide how many bits, which structure to keep, and how many streams for each RU.**

# What the project is built from

*Three parts; matrix decomposition and matrix inversion are the common tools*

## A. Compress with matrix decomposition

- The received  $64 \times 1200$  matrix has structure: only ~23 propagation paths → low rank; sparse in the delay domain.
- Truncated SVD [repo → fix]: keep the  $r$  largest singular values (best possible rank- $r$  approximation).
- RAS-BFP [new]: randomised SVD — sketch, QR, small SVD —  $\approx 5\times$  faster than the thin SVD.
- CSEE [new]: IFFT along subcarriers, keep the  $K$  strongest delay taps.
- BFP [repo]: the O-RAN standard quantiser, used as the baseline and inside every encoder.
- ACAFS [fix]: estimate the rank and send only that many streams.

## B. Cost of matrix inversion

- The receiver needs  $A^{-1}$  (MMSE equaliser, channel estimation) every slot. Where can it run — RU or DU?
- Benchmark [repo, re-run]: LU, QR, SVD, Gauss–Jordan, Newton–Schulz vs three neural 'InverseNet' models.
- Dataset: random well-conditioned matrices,  $n = 10 / 100 / 500$ , float32; 80 % train / 20 % held out.
- Learned models: direct MLP, learned Newton–Schulz (17 parameters), InverseNet-Ultra.
- Fixed [fix]: NumPy inference engine used the wrong activation; overstated docstring.

## C. Share one link

- Each RU offers a menu of settings (bits, taps, rank), each with a known rate and a predicted error.
- Allocator [new]: pick one setting per RU so the total rate fits the link and the total error is smallest.
- Prediction comes from the decomposition itself (energy of discarded taps / singular values) — no trial encoding.
- Compared with static settings and with an oracle that encodes every option.

# Starting point: what the repository contained and what I fixed

*Audit done on 17 Sep 2026 (details in docs/audit.md)*

| Part                                         | Found           | Problem found                                                                                                     | Now                             |
|----------------------------------------------|-----------------|-------------------------------------------------------------------------------------------------------------------|---------------------------------|
| Fronthaul experiments 1-4, notebook, figures | [repo] present  | they import a package `src/` that was never committed → nothing could run; figures not reproducible               | [new] `src/` rebuilt, 42 tests  |
| SVD baseline timing (Exp2)                   | [repo] figure   | the full 1200×1200 SVD was timed (~1.7 s); the thin SVD takes 13 ms → 'speed-up vs SVD' was ~100x too high        | [fix] thin SVD                  |
| Functional-split model (Exp3)                | [repo] figure   | split 6 treated as the most expensive; 3GPP says it is the cheapest. 'Fast rank' curve was exact rank + random ±1 | [fix] model + real estimator    |
| Error vs SNR (Exp4)                          | [repo] figure   | error measured against the noisy matrix follows −SNR by construction; text called it 'stable'                     | [fix] two references            |
| CSEE input                                   | [repo] docs     | tested only on unmodulated symbols; real data symbols destroy the delay sparsity                                  | [new] data vs reference symbols |
| Sharing one link                             | absent          | the actual 'load management' decision did not exist                                                               | [new] allocator + Exp5          |
| Matrix-inversion benchmark                   | [repo] complete | NumPy engine used ReLU while the trained model uses GELU; docstring overstated results                            | [fix] + re-run                  |

**Order of work: (1) make it run and make it correct, (2) fair baselines, (3) one new, well-tested piece — the allocator.**

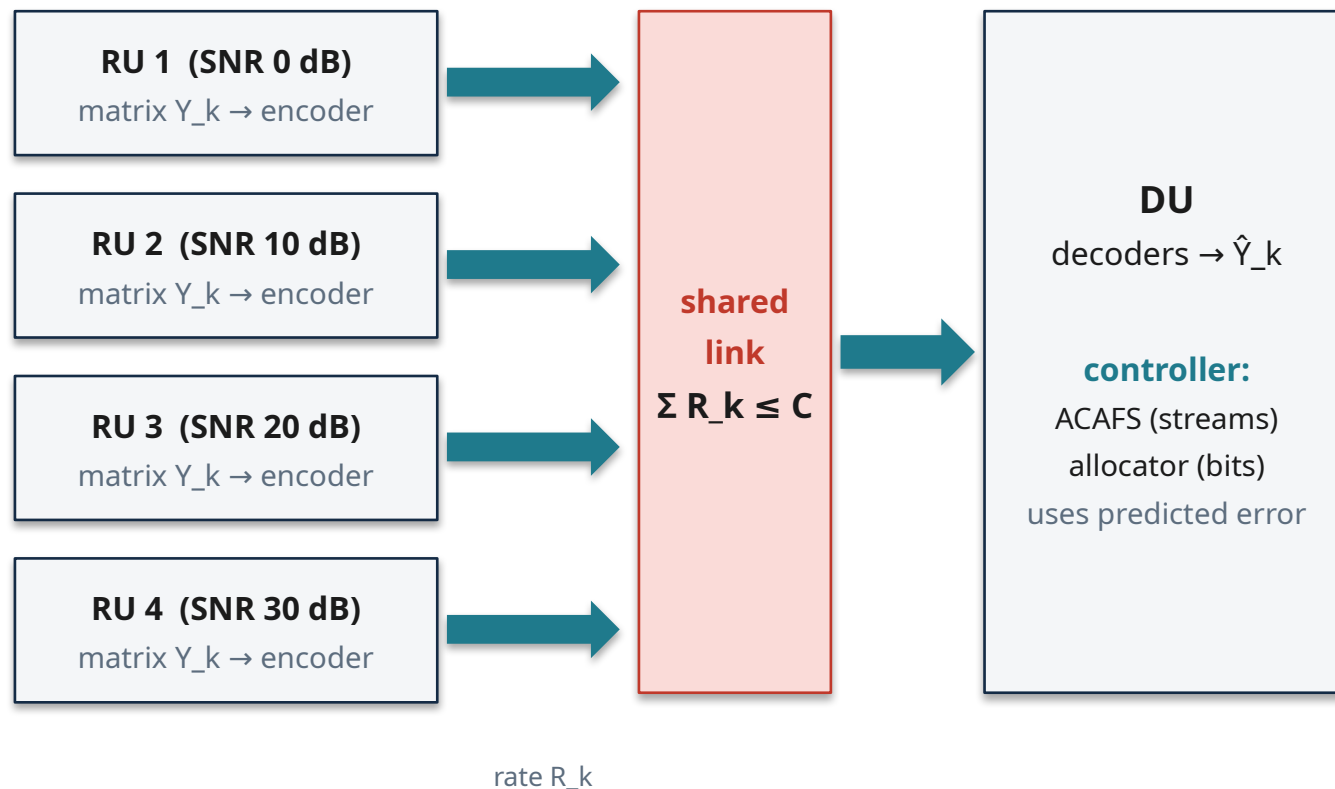
# Research context and the gap this project addresses

*Checked sources only; full list with links in docs/research\_review.md (search cutoff 17 Sep 2026)*

| Direction                                             | Representative work                                                                                              | What it assumes / measures                                                                                                                     | Relation to this project                                                                                                              |
|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Standardised I/Q compression                          | O-RAN WG4 CUS Annex A [5]; Silva et al., JCIS 2022 [6]                                                           | per-PRB BFP / block scaling / $\mu$ -law, 4-bit exponent; SQNR vs iqWidth, single RU                                                           | our BFP baseline reproduces this bit accounting; their per-PRB method selection $\approx$ our per-RU menu, but no capacity constraint |
| Fronthaul survey / functional splits                  | Park et al., IEEE SPM 2014 [1]; Larsen et al., COMST 2019 [3]; 3GPP TR 38.801 Annex A [4]                        | quantize-and-forward theory; UL bit-rate per split: Opt.6 $\approx$ 5.6 Gb/s $\ll$ Opt.7b 54-86 $\ll$ Opt.8 157 Gb/s                           | source of the split-ordering correction in ACAFS; motivates rank-adaptive stream count                                                |
| Shared fronthaul, centralised control                 | Lagén et al., IEEE ComMag 2021/2022 [7][9]; IEEE TMC 2023 [8]                                                    | several cells on one FH link; DL modulation compression + MAC scheduling; ns-3 throughput                                                      | template for Exp5 (per-cell operating point, common capacity); ours is UL I/Q with NMSE, no MAC                                       |
| Structured compression of channel / received matrices | Wen et al., WCL 2018 (CsiNet) [15]; Qiao-Jiang-Yu, ICC 2023 / TWC 2024 [13][14]; Rahmani et al., arXiv 2025 [10] | delay-domain sparsity of H; learned dimension-reduction matrices from local CSI; O-RAN BFP under TDL, BLER                                     | explains why CSEE works only on reference symbols; low-rank sketch = our RAS-BFP; BLER is the metric we still lack                    |
| Closest single-link prior art                         | Aswathylakshmi-Ganti, WCNC 2022 [22]; Li et al., IEEE Access 2019 [24]; Kanno et al., VTC 2023 [25]              | 64-ant. TDL-A delay-domain low rank via blind deconvolution; bit-width vs load/SINR on one eCPRI link; SVD streams + bits per stream at one AP | CSEE's structure, but data symbols removed first — hence CSEE needs reference symbols; allocator extends [24][25] to many RUs         |
| Learning-based split orchestration                    | Murti et al., EuCNC 2022 [16]; IEEE TNSM 2024 [17]                                                               | DQN / D3QN over minutes-scale demand traces; cost model from testbed; up to 59-69 % cost saving vs static                                      | justifies adaptive split as a topic; RL not warranted here because our rate model is closed-form per symbol                           |
| Numerical foundations                                 | Halko-Martinsson-Tropp, SIAM Rev. 2011 [19]; Tropp 2011 [20]; Shoham-Gersho 1988 [21]                            | randomised range finder + Frobenius bound; SRHT subspace embedding; $\lambda$ -sweep / convex-hull bit allocation                              | RAS-BFP algorithm and Thm-2 bound; allocator = Shoham-Gersho on RU menus (no algorithmic novelty claimed)                             |

**Gap addressed: prior art adapts bits or streams on one link [22][24][25]; here a closed-form per-symbol predictor drives allocation across RUs sharing one link, with a metric separating signal loss from removed noise.**

3GPP TR 38.901 TDL-A uplink channel, one OFDM symbol per RU



$Y = H \cdot \text{diag}(x) + W$  (64 antennas  $\times$  1200 subcarriers)

H: channel. 23 paths (TDL-A taps), 100 ns delay spread, antenna correlation  $0.7^{|i-j|}$ .

So  $\text{rank}(H) \leq 23$ , and an IFFT over the subcarriers puts almost all energy in a few delay taps.

x: the transmitted symbols. Data symbols = random QPSK per subcarrier. Reference symbols = known, so the RU can divide them out and is left with  $H + \text{noise}$ .

W: white noise at the chosen SNR.

## What we measure

Error: NMSE (dB) =  $10 \log_{10} \|Y - \hat{Y}\|^2 / \|Y\|^2$  — against the noisy  $Y$  the link carries, and against the clean  $H$ .

Compression ratio = 16-bit size / bits actually sent (exponents and headers counted).

Encode time (Python medians; only the ordering is meaningful).

For the allocator: link utilisation, RUs dropped, predicted-vs-measured error, decision time.

# Compression = decompose the matrix, keep the big part, quantise

[new]

*Every encoder ends with O-RAN block floating point (12-sample blocks, 4-bit shared exponent)*

| Encoder                   | Decomposition used                                                                                                    | What is sent                                                    | Cost                     | Error model                               |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|--------------------------|-------------------------------------------|
| <b>BFP (O-RAN WG4)</b>    | none — only quantisation                                                                                              | all $M \cdot N$ samples at $b$ bits + exponents                 | $O(MN)$                  | quantisation-noise formula                |
| <b>Truncated SVD</b>      | $Y \approx U_r \Sigma_r V_r^H$ (best rank- $r$ approximation)                                                         | $\text{BFP}(U_r \Sigma_r) + \text{BFP}(V_r)$                    | $O(MN \cdot M)$ thin SVD | Eckart-Young: sum of dropped $\sigma_i^2$ |
| <b>RAS-BFP (proposed)</b> | randomised SVD: sketch antennas $\rightarrow$ QR $\rightarrow$ project $\rightarrow$ small SVD $\rightarrow$ rank $r$ | same shape as SVD                                               | $O(MN(r+p))$             | HMT-type bound (Thm 2)                    |
| <b>CSEE (proposed)</b>    | IFFT along subcarriers $\rightarrow$ keep the $K$ strongest delay taps (same taps for all antennas)                   | $\text{BFP}(64 \times K \text{ block}) + K \text{ tap indices}$ | $O(MN \log N)$           | Prop. 1: dropped-tap energy is exact      |

## • Which structure survives the transmitted symbols? $Y = H \cdot \text{diag}(x) + W$

- Rank: multiplying by  $\text{diag}(x)$  (an invertible matrix) does not change the rank  $\rightarrow$  SVD and RAS-BFP work on data and reference symbols alike.
- Delay sparsity:  $\text{IFFT}(H \cdot \text{diag}(x)) = \text{IFFT}(H)$  convolved with  $\text{IFFT}(x)$ . Random symbols make  $\text{IFFT}(x)$  white, so the energy spreads over all 1200 taps  $\rightarrow$  CSEE only works on reference symbols.
- The earlier repository did not see this because it simulated a constant symbol on every subcarrier.

# How the shared link is allocated, and why it is cheap

[new]

*One decision per OFDM symbol: a multiple-choice knapsack*

## The decision

Each RU  $k$  picks one setting  $a_k$  from its menu (for CSEE:  $K$  taps and  $b$  bits).

Every setting has an exact rate  $R_k(a)$  in bits and an error  $D_k(a) = \|Y_k - \hat{Y}_k\|^2 / \|Y_k\|^2$ .

**minimise  $\sum_k D_k(a_k)$  [or the worst RU:  $\max_k D_k(a_k)$ ]**

**subject to  $\sum_k R_k(a_k) \leq C_{\text{link}}$**

Solver: start every RU at its cheapest setting, then repeatedly give the next upgrade to the RU that gains the most error reduction per bit (on the convex hull of its menu). Optimal up to one fractional step. If even the cheapest settings do not fit, drop the RUs that gain least.

Measured:  $\approx 1$  ms to allocate +  $\approx 27$  ms to predict all menus for 8 RUs  $\times$  66 settings (Python).

## Proposition 1: predicting CSEE's error without encoding

Take the IFFT once:  $Y_d$ . Sort the delay taps by energy.

**error = ( energy of dropped taps + quantisation error ) / total energy**

The first term is exact (the IFFT preserves energy). The second comes from the block exponents:  $\Delta^2/6$  per sample on average,  $\Delta^2/2$  worst case.

Noise-aware version: subtract the noise that is expected in the dropped taps and divide by signal energy only — uses the RU's noise estimate  $\sigma^2$ .

Because the top- $K$  sets are nested, one IFFT gives the prediction for every  $K$  and every  $b$ .

**Measured: prediction within 0.5 dB of the real error for CSEE in Exp5 (BFP predictor within 1.3 dB).**

## Theorem 2 (RAS-BFP): reference bound from Halko–Martinsson–Tropp 2011, Thm 10.5

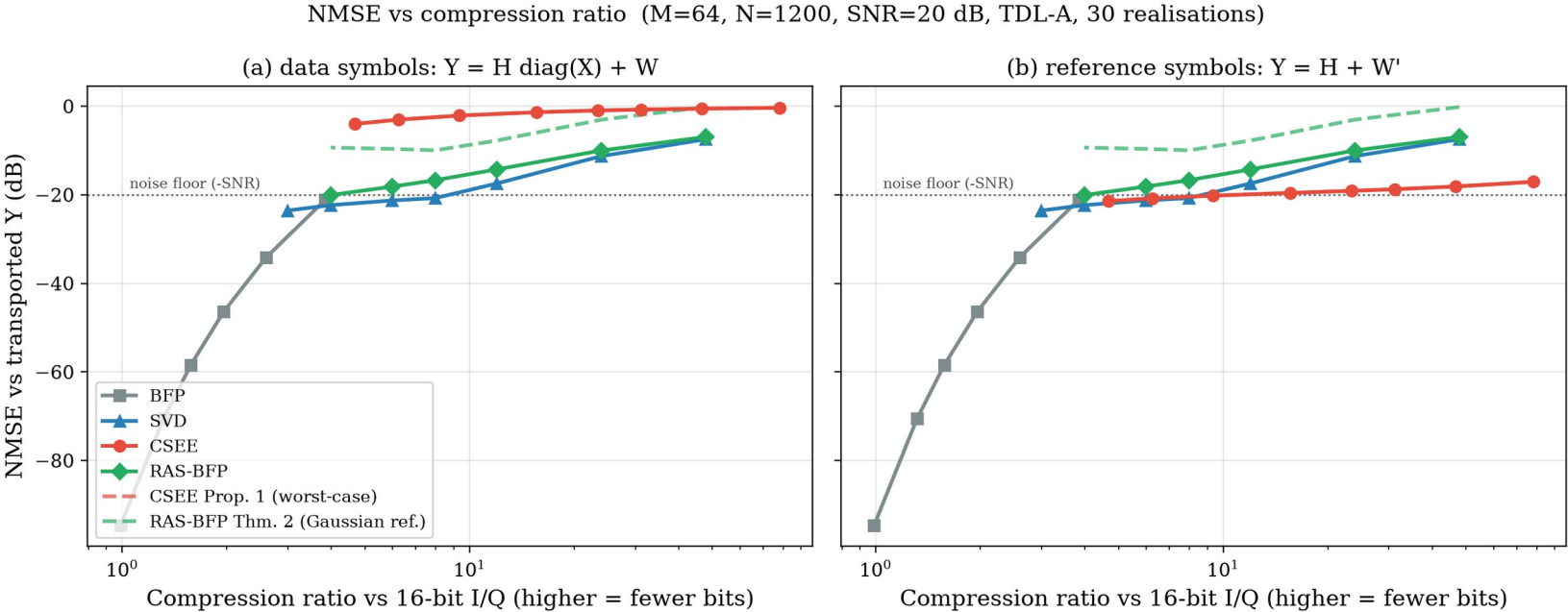
**$E\|Y - \hat{Y}_r\|_F^2 \leq (4 + 2r/(p-1)) \cdot \sum_{i>r} \sigma_i^2$  (Gaussian sketch,  $p \geq 2$ )**



# Result 1 — how much compression for how much error (Exp1)

[new]

$M=64, N=1200, SNR\ 20\ dB, 30\ channel\ realisations; b = 10\ bits\ for\ SVD / CSEE / RAS-BFP$



| Setting        | CR    | data     | reference |
|----------------|-------|----------|-----------|
| BFP $b=8$      | 2.0x  | -46.4 dB | -46.4 dB  |
| SVD $r=12$     | 8.0x  | -20.7 dB | -20.7 dB  |
| RAS-BFP $r=12$ | 8.0x  | -16.7 dB | -16.7 dB  |
| CSEE $K=120$   | 15.6x | -1.3 dB  | -19.6 dB  |

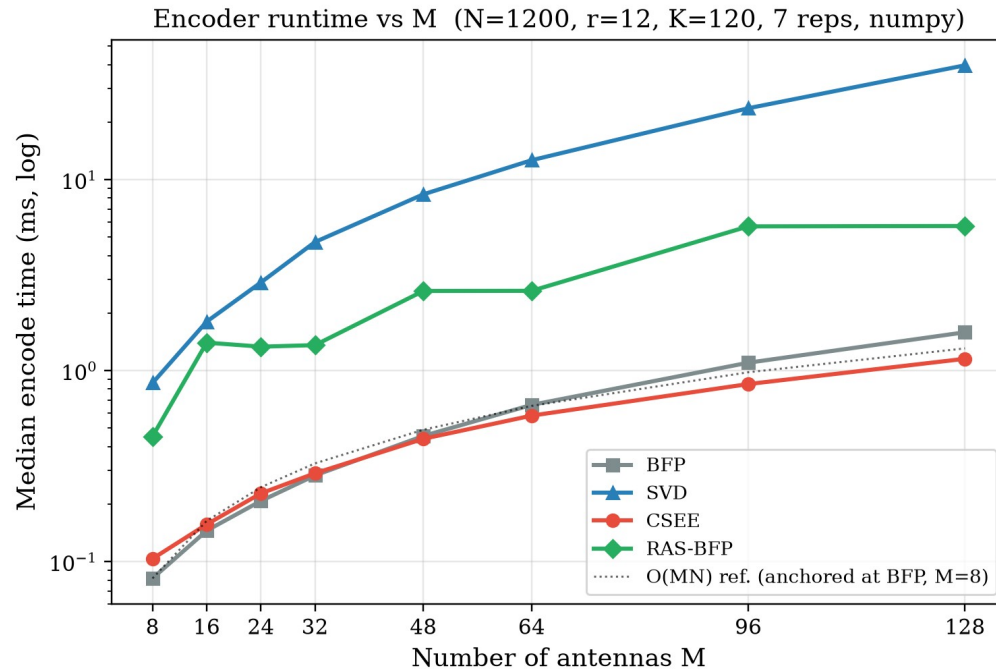
- Error here = NMSE against the transported (noisy) matrix Y; lower is better.
- CSEE:  $-17$  to  $-21$  dB at 6–78x on reference symbols, but about 0 dB (useless) on data symbols.
- RAS-BFP costs 1–4 dB more than SVD for the same payload; the Thm-2 bound sits 6–10 dB above it (valid, loose).
- Every truncating encoder flattens at  $-20$  dB: that is the noise floor of this metric at 20 dB SNR.

CSEE compresses channel (reference-symbol) matrices, not general data; low-rank methods (SVD, RAS-BFP) work for both.

# Results 2-3 — encoding time, and two ways to measure error

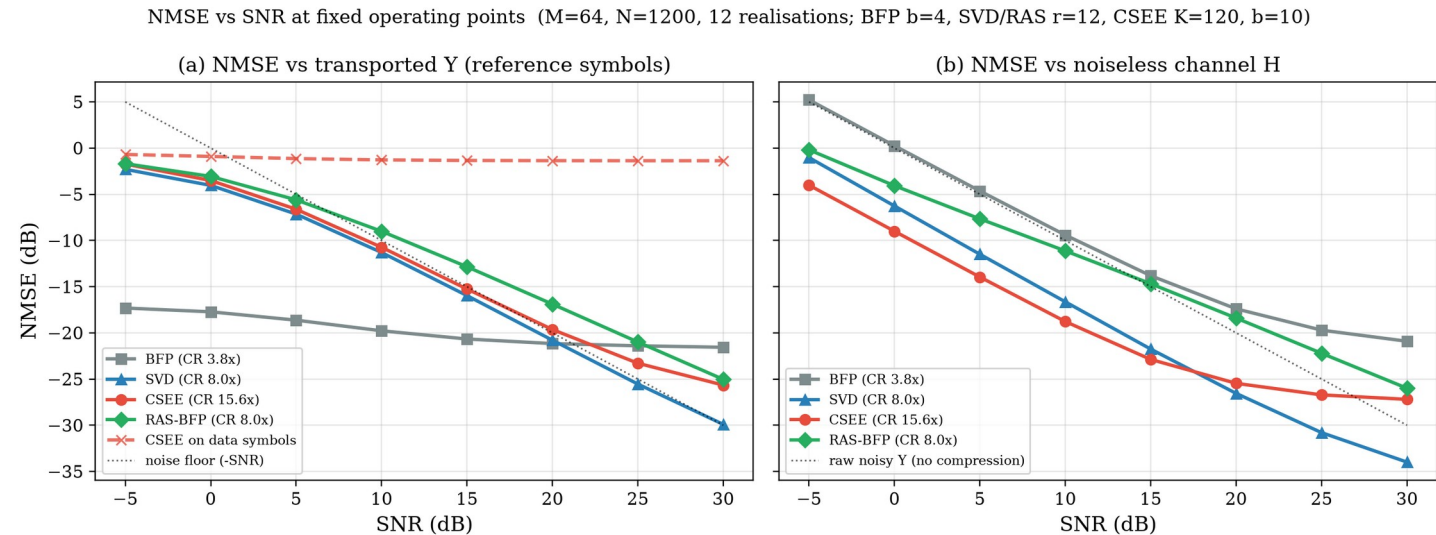
[new]

Exp2: Python medians of 7 runs,  $N=1200$  · Exp4: reference symbols, settings at  $CR \approx 8x$ , 12 realisations



**M=64: BFP 0.66 ms · CSEE 0.58 ms · RAS-BFP 2.6 ms · thin SVD 12.6 ms**

Randomised SVD is  $\approx 5x$  faster than the thin SVD at  $M=64$  and  $\approx 7x$  at  $M=128$  (the old figure said  $\sim 100x$ , against a full SVD)



(a) against noisy Y: truncating encoders follow the  $-SNR$  line. (b) against clean H: at 0 dB SNR the raw matrix is at 0 dB, CSEE at -9 dB, SVD at -6 dB — the decompositions remove noise.

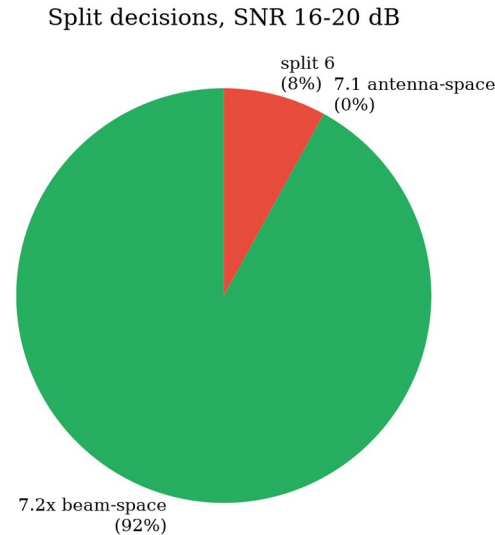
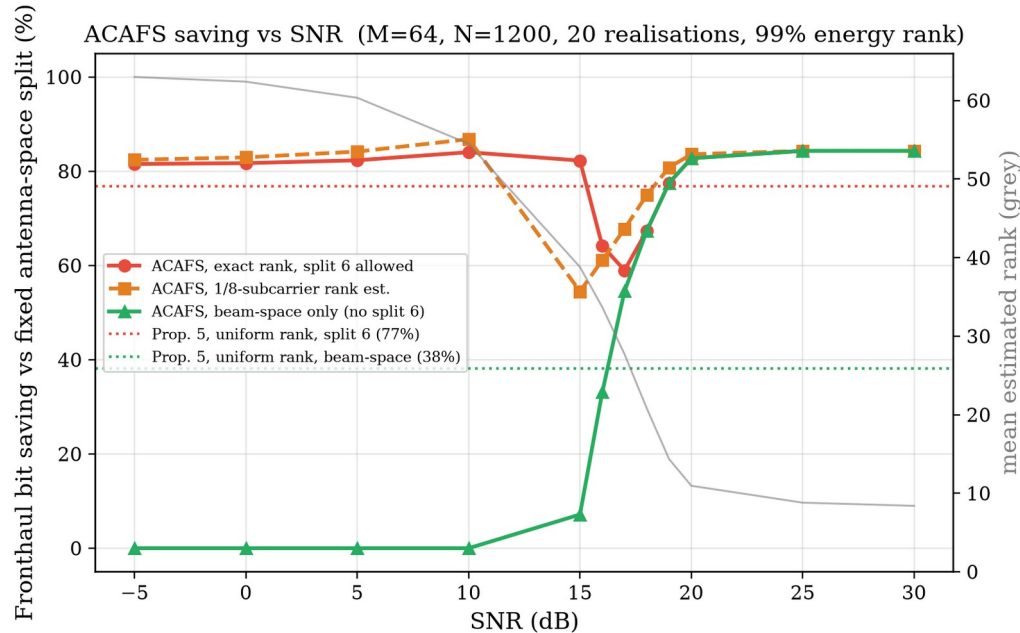
At 30 dB: SVD -34, CSEE -27, RAS-BFP -26, BFP  $b=4$  -21 dB. CSEE on data symbols (dashed) stays at  $\approx -1$  dB.

'Stable across SNR' in the old report was an artefact of the metric. Against the clean channel, the decompositions act as denoisers at low SNR.

# Result 4 — send only as many streams as the rank (ACAFS, Exp3)

[fix]

$M=64$ ,  $N=1200$ , 20 realisations per SNR; rank = number of eigenvalues of  $Y Y^H$  holding 99 % of the energy



## • Corrected cost model (3GPP TR 38.801 / O-RAN)

- all antennas (split 7.1 / 7.2x-A):  $2MNb$  bits — the reference
- $r$  streams (7.2x-B beam-space):  $2rNb + \text{weights}$
- split 6 (decoded bits):  $r \cdot N \cdot 6$  bits  $\rightarrow \approx 20x$  cheaper, but the RU must run the whole uplink receiver
- Rule: if  $\text{rank}/M < 0.55 \rightarrow$  send  $\max(\text{rank}, 10)$  streams; else split 6 if the RU has the compute, else all antennas.

## • Streams only: 0 % saving below 15 dB $\rightarrow$ 83 % at 20 dB (mean rank 11).

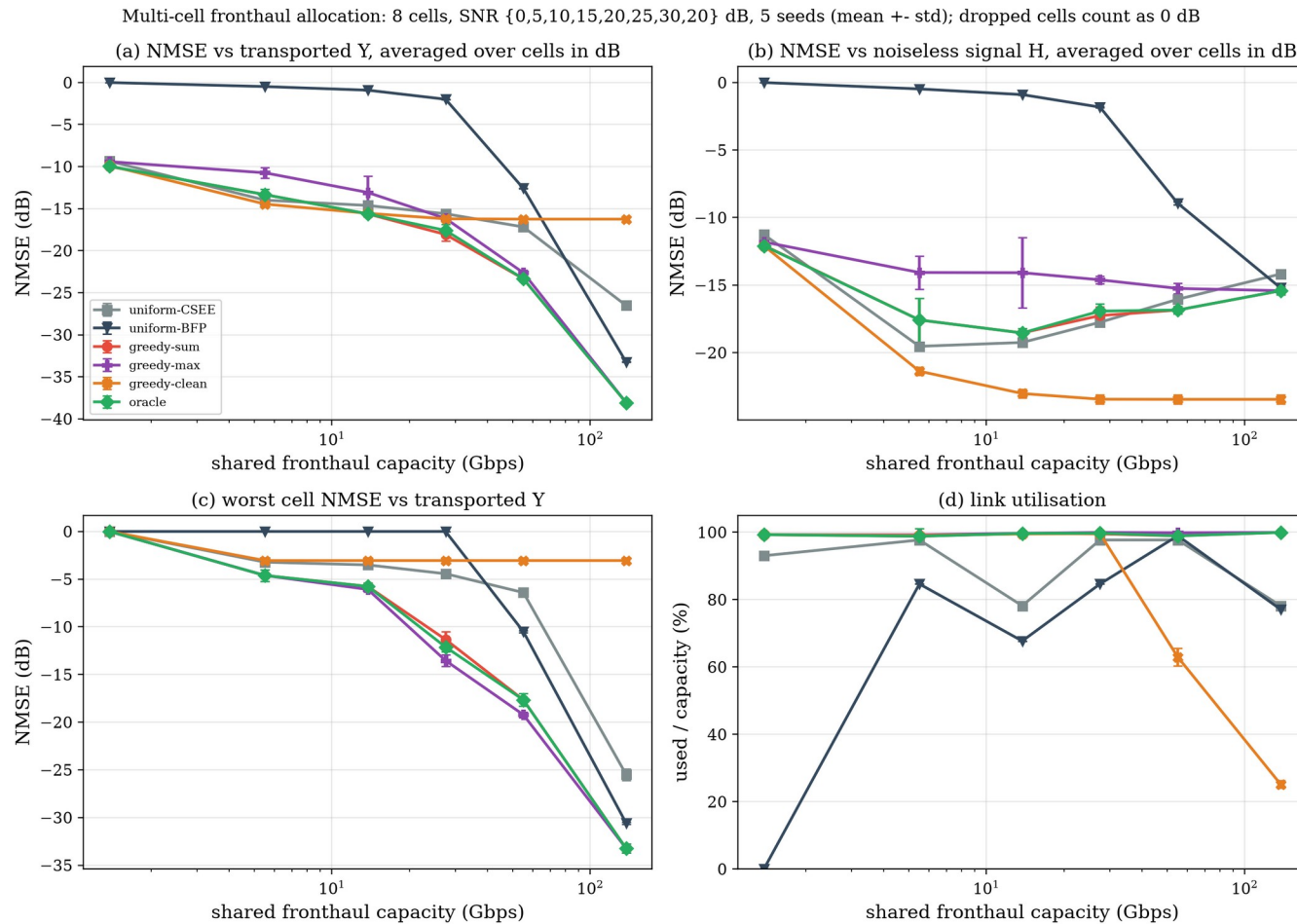
- Fast rank estimate (1/8 of the subcarriers): 31 % less time, but under-estimates the rank by up to 10 at 10–17 dB  $\rightarrow$  the 10-stream floor protects against it.
- Prop. 5 (assumes every rank equally likely): 38 % streams-only, 77 % with split 6 — a prediction, not a bound.

The old 'saving vs split 6' figure had the costs upside down. Savings come from the rank collapsing at high SNR, and from whether the RU may host split 6.

# Result 5 — sharing one link between 8 RUs (Exp5)

[new]

*RU SNRs {0,5,10,15,20,25,30,20} dB · menu of 11  $K \times 6 b = 66$  CSEE settings · 5 seeds · reference symbols*



## • Policies compared

- uniform-CSEE / uniform-BFP: same setting for every RU (today's static configuration)
- greedy-sum / greedy-max: proposed; decides from Prop.-1 predictions only
- greedy-clean: proposed; noise-aware objective (error of the signal, not of the noise)
- oracle: greedy on measured error — encodes every option first (ablation)

## • What the panels show

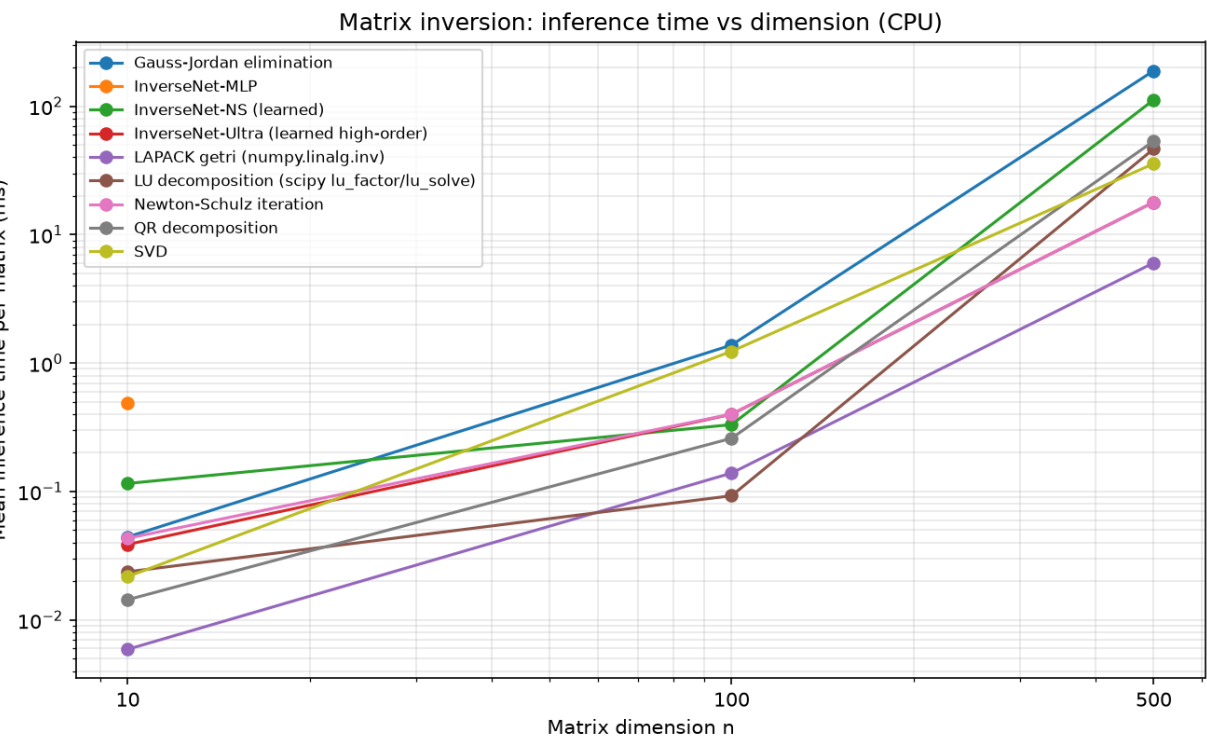
- (a) vs Y: greedy is within 0.5 dB of the oracle everywhere; 2.5 dB better than uniform at 27.5 Gbps, 6.2 dB at 55 Gbps; uniform-BFP does not fit below 55 Gbps (RUs dropped)
- (b) vs H: greedy-sum spends bits on reproducing noise in the low-SNR RUs; the noise-aware objective is 6.2 dB better at 27.5 Gbps
- (c) min-max objective: worst RU up to 2.2 dB better (27.5 Gbps) at a 0.7–2.6 dB cost in the mean
- (d) the noise-aware policy uses only 25 % of the link at 138 Gbps — more bits would only copy noise

**Predicting the error is enough to allocate as well as encoding everything. The objective matters: minimise loss of signal, not loss of noise.**

# Result 6 — how much does one matrix inversion cost?

[repo]

Why it matters: MMSE equalisation and channel estimation invert a matrix per resource-block group every slot; split 6 puts that inside the RU



| n   | LAPACK LU<br>(numpy.linalg.inv) | Newton-Schulz | thin SVD | Gauss-Jordan |
|-----|---------------------------------|---------------|----------|--------------|
| 10  | 0.00581 ms                      | 0.0423 ms     | 0.021 ms | 0.0439 ms    |
| 100 | 0.138 ms                        | 0.403 ms      | 1.23 ms  | 1.38 ms      |
| 500 | 5.86 ms                         | 18.1 ms       | 35.7 ms  | 189 ms       |

- Set-up: random well-conditioned matrices  $A = G/\sqrt{n} + 2I$ , float32,  $n = 10 / 100 / 500$ ; 200 held-out matrices (20 at  $n=500$ ); 4-core CPU, one matrix at a time.
- LAPACK LU is fastest at every size and exact to float32 (relative error  $\approx 5e-8$ ).
- **A 100×100 inverse takes  $\approx 0.14$  ms → about 7 per core fit in a 1 ms slot (15 kHz spacing). A 500×500 inverse takes  $\approx 6$  ms → it does not fit.**
- Iterative Newton-Schulz is 3-7x slower than LU but needs only matrix multiplications — attractive for hardware, and the basis of the learned models on the next slide.
- Gauss-Jordan (textbook method) is 30x slower at  $n=500$ .

**Inversion cost bounds where processing can live: massive-MIMO-sized inverses fit a slot on a CPU core; multi-cell-sized ones do not.**

# Result 6b — can a neural network learn to invert a matrix?

[fix]

Three 'InverseNet' models, trained on the first 80 % of the dataset, tested on the last 20 %

| n   | LAPACK LU                       | InverseNet-MLP (direct regression) | InverseNet-NS (learned Newton–Schulz, 17 params) | InverseNet-Ultra (learned NS + data-dependent start) |
|-----|---------------------------------|------------------------------------|--------------------------------------------------|------------------------------------------------------|
| 10  | 0.00581 ms · rel. error 4.8e-08 | 0.0501 ms · rel. error 3.6e-01     | 0.114 ms · rel. error 4.0e-02                    | 0.0383 ms · rel. error 1.7e-01                       |
| 100 | 0.138 ms · rel. error 4.8e-08   | not run                            | 0.328 ms · rel. error 4.0e-04                    | 0.397 ms · rel. error 1.1e-06                        |
| 500 | 5.86 ms · rel. error 4.8e-08    | not run                            | 117 ms · rel. error 7.4e-03                      | 17.9 ms · rel. error 1.3e-06                         |

- MLP: flatten  $A$ , regress  $A^{-1}$ . Fails (error 0.36 at  $n=10$ ) and cannot scale — at  $n=500$  it would need  $> 2$  billion parameters.
- Learned Newton–Schulz: unroll  $X \leftarrow X(\beta I - \gamma AX)$  for 8 steps and learn the 17 scalars. It generalises to any  $n$  (error  $4e-4$  at  $n=100$ ), but it is slower than LU.
- Ultra: adds a tiny network that sets the starting guess from four matrix statistics. Error  $\approx 1e-6$  at  $n=100/500$  (but 0.17 at  $n=10$ ); same speed as classical Newton–Schulz; never beats LU medians.
- Fixed this iteration [fix]: the fast NumPy inference engine used ReLU while the trained model uses GELU — outputs now match (tested); a docstring claiming  $1e-7$  error and 'faster than LU' was corrected; benchmark re-run.
- **Conclusion: for well-conditioned matrices, learning the inverse does not beat LAPACK on a CPU. The learned iteration is a reasonable idea for fixed-latency hardware, not a replacement for LU.**

**The ML part of the project gives a clear negative result: a learned inverter does not beat classical linear algebra here — and saying so is the honest contribution.**



# What is done, what is not, and what comes next

## • Done

[new] • Working, tested simulator: channel → decomposition-based encoders → split → allocation (was missing)

[fix] • Fair baselines: thin SVD, correct split costs, real rank estimator, two error references

[new] • Error predictor within 0.5 dB; HMT bound checked

[new] • Allocator for 8 RUs with static baselines, oracle and objective ablations, 5 seeds

[new] • Finding: delay-domain compression works only on channel/reference symbols; low rank works on everything

[fix] • Matrix-inversion benchmark: bug fixed, re-run, honest conclusion (LU wins)

## • Limits

• One OFDM symbol at a time; no scheduler, retransmissions or latency

• CSEE valid on reference symbols only → Exp5 covers that traffic only

• Python timings: ordering and scaling only

• Thm 2 proven for Gaussian sketches; Hadamard sketch checked empirically

• Split-6 cost is a rough model (6 bit/RE); noise variance assumed known in the noise-aware test

• Inversion benchmark: well-conditioned random matrices, not channel matrices

• *No novelty claimed for any single algorithm; the contribution is the validated system and its findings*

## • Next, until the final evaluation

[future] • Slot-level simulation mixing data and reference symbols; menu with CSEE, RAS-BFP and BFP (needs a RAS-BFP error predictor)

[future] • Joint split + allocation under one capacity, with the inversion benchmark as the RU compute limit

[future] • Sensitivity to noise-estimate and rank errors

[future] • Inversion benchmark on real channel matrices (ill-conditioned) and on fixed-latency hardware

[future] • Learned components only where no formula exists, judged against the formula

# References (1/2) — checked sources, see docs/research\_review.md

- [1] S.-H. Park, O. Simeone, O. Sahin, S. Shamai, "Fronthaul compression for cloud radio access networks," IEEE Signal Process. Mag., vol. 31, no. 6, 2014. doi:10.1109/MSP.2014.2330031 (abstract)
- [2] M. Peng, C. Wang, V. Lau, H. V. Poor, "Fronthaul-constrained cloud radio access networks: Insights and challenges," IEEE Wireless Commun., vol. 22, no. 2, 2015. (full text, author copy)
- [3] L. M. P. Larsen, A. Checko, H. L. Christiansen, "A survey of the functional splits proposed for 5G mobile crosshaul networks," IEEE Commun. Surveys Tuts., vol. 21, no. 1, 2019. (full text)
- [4] 3GPP TR 38.801 (Rel-14), "Study on new radio access technology: Radio access architecture and interfaces," §11.1.2, Annex A Tables A-1/A-2.
- [5] O-RAN Alliance WG4, "Control, User and Synchronization Plane Specification," O-RAN.WG4.TS.CUS.0-R004-v19.00, Annex A (compression methods). (annex listing; BFP details cross-checked with [6])
- [6] M. D. L. Silva, L. Ramalho, I. Almeida, E. Medeiros, M. Berg, A. Klautau, "A new O-RAN compression approach for improved performance on uplink signals," J. Commun. Inf. Syst., vol. 37, no. 1, pp. 30–41, 2022. doi:10.14209/jcis.2022.3 (full text)
- [7] S. Lagén, L. Giupponi, A. Hansson, X. Gelabert, "Modulation compression in next generation RAN: Air interface and fronthaul trade-offs," IEEE Commun. Mag., vol. 59, no. 1, 2021. doi:10.1109/MCOM.001.2000453 (full text)
- [8] S. Lagén, X. Gelabert, L. Giupponi, A. Hansson, "Fronthaul-aware scheduling strategies for dynamic modulation compression in next generation RANs," IEEE Trans. Mobile Comput., vol. 22, no. 5, pp. 2725–2740, 2023. (abstract)
- [9] S. Lagén, X. Gelabert, A. Hansson, M. Requena, L. Giupponi, "Fronthaul compression control for shared fronthaul access networks," IEEE Commun. Mag., vol. 60, no. 6, 2022. doi:10.1109/MCOM.001.2100959 (full text)
- [10] M. Rahmani et al., "Exploring O-RAN compression techniques in decentralized distributed MIMO systems: Reducing fronthaul load," arXiv:2507.04997, 2025. (preprint, full text)
- [11] "Hardware implementation for O-RAN block floating point compression and decompression methods in 5G base station," Proc. ATC 2024. doi:10.1109/ATC63255.2024.10908174 (abstract)
- [12] S. Yagi et al., "Deep learning-based nonlinear quantizer for fronthaul compression," Proc. OECC/PSC 2022. (abstract)
- [13] R. Qiao, T. Jiang, W. Yu, "Learning-based fronthaul compression for uplink cloud radio access networks," Proc. IEEE ICC 2023, pp. 5928–5933. (abstract)
- [14] R. Qiao, T. Jiang, W. Yu, "Meta-learning-based fronthaul compression for cloud radio access networks," arXiv:2403.09004, 2024; IEEE Trans. Wireless Commun., vol. 23, no. 9, 2024. (abstract)
- [15] C.-K. Wen, W.-T. Shih, S. Jin, "Deep learning for massive MIMO CSI feedback," IEEE Wireless Commun. Lett., vol. 7, no. 5, pp. 748–751, 2018. (full text)



## References (2/2) — checked sources, see docs/research\_review.md

- [16] F. W. Murti, S. Ali, G. Iosifidis, M. Latva-aho, "Learning-based orchestration for dynamic functional split and resource allocation in vRANs," Proc. EuCNC/6G Summit 2022. (full text)
- [17] F. W. Murti, S. Ali, G. Iosifidis, M. Latva-aho, "Deep reinforcement learning for orchestrating cost-aware reconfigurations of vRANs," IEEE Trans. Netw. Service Manag., vol. 21, no. 1, pp. 200–216, 2024. doi:10.1109/TNSM.2023.3292713 (full text)
- [18] F. W. Murti, S. Ali, M. Latva-aho, "Constrained deep reinforcement based functional split optimization in virtualized RANs," arXiv:2106.00011, 2021. (abstract)
- [19] N. Halko, P.-G. Martinsson, J. A. Tropp, "Finding structure with randomness," SIAM Rev., vol. 53, no. 2, pp. 217–288, 2011. doi:10.1137/090771806 (full text)
- [20] J. A. Tropp, "Improved analysis of the subsampled randomized Hadamard transform," Adv. Adapt. Data Anal., vol. 3, no. 1–2, pp. 115–126, 2011. (full text, preprint)
- [21] Y. Shoham, A. Gersho, "Efficient bit allocation for an arbitrary set of quantizers," IEEE Trans. Acoust., Speech, Signal Process., vol. 36, no. 9, pp. 1445–1453, 1988. doi:10.1109/29.90373 (abstract)
- [22] Aswathylakshmi P, R. K. Ganti, "Fronthaul compression for uplink massive MIMO using matrix decomposition," Proc. IEEE WCNC 2022, pp. 2524–2529. doi:10.1109/WCNC51071.2022.9771783 (full text)
- [23] F. Wiffen, W. H. Chin, A. Doufexi, "Distributed dimension reduction for distributed massive MIMO C-RAN with finite fronthaul capacity," Proc. Asilomar 2021, pp. 1228–1236. doi:10.1109/IEEECONF53345.2021.9723180 (full text)
- [24] L. Li et al., "Enabling flexible link capacity for eCPRI-based fronthaul with load-adaptive quantization resolution," IEEE Access, vol. 7, pp. 102174–102185, 2019. doi:10.1109/ACCESS.2019.2930214 (full text)
- [25] I. Kanno et al., "Adaptive bit allocation for SVD based hybrid processing of uplink cell-free massive MIMO under limited fronthaul capacity," Proc. IEEE VTC2023-Spring. doi:10.1109/VTC2023-Spring57618.2023.10201009 (abstract)
- [26] L. Liu, R. Zhang, "Optimized uplink transmission in multi-antenna C-RAN with spatial compression and forward," IEEE Trans. Signal Process., vol. 63, no. 19, pp. 5083–5095, 2015. (abstract)
- [27] A. Martínez Alba, W. Kellerer, "Dynamic functional split adaptation in next-generation radio access networks," IEEE Trans. Netw. Service Manag., vol. 19, no. 3, pp. 3239–3263, 2022. (abstract)
- [28] L. Wang, S. Zhou, "On the fronthaul statistical multiplexing gain," IEEE Commun. Lett., vol. 21, no. 5, pp. 1099–1102, 2017. (abstract)
- [29] 3GPP TR 38.901 V19.4.0, "Study on channel model for frequencies from 0.5 to 100 GHz," cl. 7.7.2–7.7.5 (TDL-A, delay scaling, correlation extension).

# Backup — audit: problems found in the original material

- ``src/`` absent from every commit and branch; Exp1-4 and the notebook import it → nothing ran
- Exp2 SVD baseline: committed figure  $\sim 1.7$  s at  $M=64$  vs 13 ms for the thin SVD;  $O(MN)$  reference line plotted in seconds on a millisecond axis
- Exp3: 'fast rank estimate' = exact rank + random  $\{-1,0,1\}$ ; 'Theorem 5 bound' lay below the simulated curve → not a bound
- ACAFS: split 6 treated as the most expensive split; TR 38.801 ordering is  $R_6 \ll R_{7.2x\text{-beam}} \leq R_{7.x\text{-antenna}}$
- Exp4: 'stable across SNR' claimed while the figure shows  $NMSE \approx -SNR$
- CSEE input: constant pilot  $\Rightarrow$  delay sparsity; with random QPSK per subcarrier CSEE gives  $\approx -0.5$  dB
- CR reference 32-bit components  $\Rightarrow$  all CRs inflated 2x (BFP  $b=8$ : 3.9x quoted vs 2.0x against 16-bit)
- Docs: 'Theorems 1-5' without statements or proofs; '100 MHz at 15 kHz' with  $N=1200$  (= 18 MHz); journal targets and hardware roadmap not backed by code
- `matinv_bench`: ReLU vs GELU in the NumPy engine; docstring claiming  $1e-7$  error and LU-beating latency
- Original figures and tables preserved in `results/original_snapshot/` for comparison

# Backup — Proposition 1 step by step, and the noise-aware form

- Set-up:  $Y$  is  $M \times N$ .  $Y_d$  = IFFT along each row (numpy convention:  $Y = \text{FFT}(Y_d)$ , so  $\|Y\|_F^2 = N\|Y_d\|_F^2$ ). Encoder keeps columns  $S$  ( $|S| = K$ ) and BFP-quantises them:  $\hat{Y}_d = Q(Y_d[:,S])$  on  $S$ , 0 elsewhere. Decoder:  $\hat{Y} = \text{FFT}(\hat{Y}_d)$ .
- $\|Y - \hat{Y}\|_F^2 = N\|Y_d - \hat{Y}_d\|_F^2 = N(\sum_{n \notin S} \|Y_d[:,n]\|^2 + \|Y_d[:,S] - Q(Y_d[:,S])\|_F^2) = N(T_S + E_q)$ .
- $\text{NMSE} = (T_S + E_q) / \|Y_d\|_F^2$ .  $T_S$  (dropped taps) is exact;  $E_q$  (quantisation) depends on the quantiser.
- BFP block with exponent  $e_b$ , step  $\Delta_b = 2^{e_b}$ : per real component  $|\text{error}| \leq \Delta_b/2$ , variance  $\approx \Delta_b^2/12$ . Per complex sample:  $\leq \Delta_b^2/2$  worst case,  $\approx \Delta_b^2/6$  expected. Sum over the non-zero samples of each block. Zero samples quantise exactly.
- Top- $K$  sets are nested in  $K$ , so one sort of the  $N$  tap energies gives  $T_S$  for all  $K$ ; block maxima per  $K$  give  $\Delta_b$  for all  $b$  by a shift of  $\log_2(2^{b-1} - 1)$ . The whole 66-setting menu costs about one encoder pass.
- Noise-aware form (RU knows  $\sigma^2$ ): white noise has total delay-domain energy  $M\sigma^2$  spread evenly over the  $N$  taps. Signal error  $\approx [\max(T_S - M\sigma^2(N-K)/N, 0) + M\sigma^2 K/N + E_q] / (\|Y_d\|_F^2 - M\sigma^2)$ . Measured within 0.1 dB at 20 dB SNR and  $\approx 1.7$  dB at 0 dB SNR.
- Why it matters: with the plain form the allocator upgrades low-SNR RUs to reproduce noise; with the noise-aware form extra bits stop paying once the dropped taps are only noise, so capacity is released.

# Backup — RAS-BFP algorithm and Theorem 2

- 1. Sketch the antenna dimension:  $Z = \Omega Y$ , size  $(r+p) \times N$ .  $\Omega = \sqrt{M/(r+p)} \cdot S \cdot H_M \cdot D$  (SRHT: random signs  $D$ , Walsh–Hadamard  $H_M$  by fast transform, row sampler  $S$ ) or a Gaussian matrix.
- 2. Thin QR:  $Z^H = Q R$ .  $Q$  ( $N \times (r+p)$ ) spans an approximate dominant right singular subspace of  $Y$ .
- 3. Project:  $L = Y Q$  ( $M \times (r+p)$ ); small SVD of  $L$ ; keep rank  $r$ :  $L_r = U_r \Sigma_r$ ,  $Q_r = Q V_r$ .  $\hat{Y} = L_r Q_r^H$  (HMT Algorithms 4.1 + 5.1).
- 4. Send BFP( $L_r$ ) and BFP( $Q_r$ ) — the same payload shape as truncated SVD at rank  $r \Rightarrow$  equal CR, fair comparison.
- Cost:  $O(MN \log M)$  sketch (SRHT) or  $O(MN(r+p))$  (Gaussian) +  $O(N(r+p)^2)$  QR +  $O(MN(r+p))$  projection, against  $O(MN \cdot M)$  for the thin SVD.
- Theorem 2 (HMT 2011, Thm 10.5, Gaussian  $\Omega$ ,  $p \geq 2$ ):  $E\|Y - YQQ^H\|_F^2 \leq (1 + r/(p-1)) \sum_{i>r} \sigma_i^2$ . Truncating to rank  $r$  adds at most an Eckart–Young term ( $\|Y - Q[Q^H Y]_r\| \leq \|Y - QQ^H Y\| + \|Y - Y_r\|$ ), so  $E\|Y - \hat{Y}\|_F^2 \leq (4 + 2r/(p-1)) \sum_{i>r} \sigma_i^2$ . Quantisation comes on top.
- Caveats: proven for Gaussian sketches; the SRHT (Tropp 2011) needs larger oversampling for a proof — with  $p = 4$  the bound holds empirically with 6–10 dB slack in Exp1. For  $p < 2$  the code returns  $\infty$  rather than an invalid bound. SRHT falls back to Gaussian when  $M$  is not a power of two or  $r+p \geq M$ .

# Backup — ACAFS cost model and allocator pseudo-code

- **Bits per OFDM symbol per RU ( $M=64$ ,  $N=1200$ ,  $b=16$ ,  $\eta=6$ ,  $T_c=14$ )**

- all antennas:  $2MNb = 2\,457\,600$
- $r$  streams:  $2rNb + 2Mrb/T_c$  ( $r=10$ : 385 463;  $r=32$ : 1 233 481)
- split 6,  $r$  layers:  $rN\eta$  ( $r=10$ : 72 000;  $r=64$ : 460 800)
- rule:  $\rho = r/M$ ;  $\rho < \tau_{\text{high}} = 0.55 \rightarrow \text{streams} = \max(r, \lceil 0.15 \cdot M \rceil = 10)$ ; else split 6 if allowed, else all antennas
- Prop. 5 = average over  $r \in \{1..M\}$  of the saving under the rule (a prediction; the simulated saving can exceed it when ranks concentrate low)

- **greedy\_allocate(rates, errors, C, objective)**

- for each RU: keep only the (R, D) points on the lower convex hull (dominated points never help)
- start every RU at its cheapest point; if  $\sum R > C$ : drop the RUs with the largest predicted error at that point until it fits
- repeat: among feasible next-hull upgrades pick the largest  $\Delta D/\Delta R$  (sum objective) or the RU with the current largest D (max objective); stop when nothing fits
- guarantee: Lagrangian optimum up to one fractional upgrade (standard for the multiple-choice knapsack greedy on convex hulls; Shoham–Gersho 1988)
- uniform\_allocate: best single point that fits for all RUs — the static-configuration baseline
- tests: capacity respected, greedy  $\geq$  uniform on its objective, low load  $\rightarrow$  best point, overload  $\rightarrow$  drops, zero-traffic RU, invalid inputs

# Backup — how to reproduce everything

- `pip install -r requirements.txt` (numpy, scipy, pandas, matplotlib, tqdm, pytest; torch only for `matinv_bench`)
- `python -m pytest tests -q` → 42 passed (~1 s)
- `python experiments/exp1_nmse_vs_cr.py` → `results/figures/exp1_*.png | pdf, results/data/exp1_nmse_vs_cr.json` (~10 s)
- `python experiments/exp2_complexity.py` → `exp2_*` (~2 s)
- `python experiments/exp3_acafs_gain.py` → `exp3_*` (~3 s)
- `python experiments/exp4_snr_sweep.py` → `exp4_*` (~5 s)
- `python experiments/exp5_fronthaul_allocation.py` → `exp5_allocation.png, exp5_allocation_runs.csv` (every run), `exp5_allocation_summary.csv` (~8 s)
- `jupyter nbconvert --execute --inplace notebooks/mid_evaluation_demo.ipynb` (run from notebooks/)
- `python -m matinv_bench.generate_dataset --dims 10 100 --num-samples 1000`; `python -m matinv_bench.benchmark --dims 10 100` (checkpoints are committed; training is optional)
- Every `results/data` file records generation time, Python/numpy version, host and CPU count; `results/logs/` has the console output of the runs shown here.
- Host used for the numbers in this deck: 4-core x86\_64, Python 3.12, numpy 2.x, no GPU.